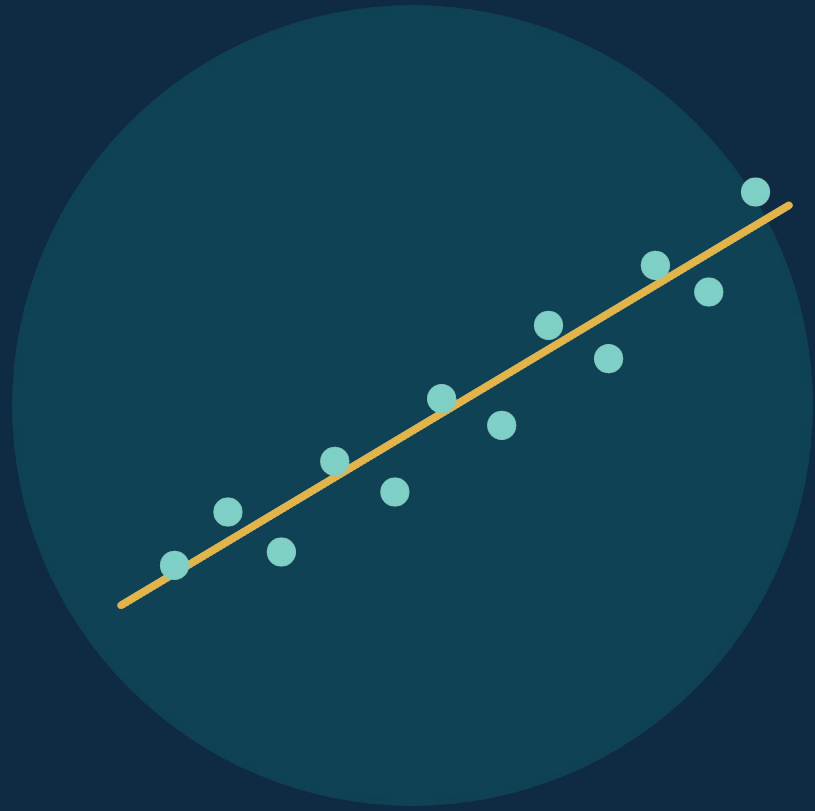
INTRODUCTION TO MACHINE LEARNING · WEEK 1

Learning from Data

Introduction to ML & Data Foundations, with Lab 1: Python Data Science & EDA

2-hour session

Lecture + Lab + 21 quiz questions



Dr.-Ing. Chukwuka Victor Obionwu

PLAN · 2 HOURS

Session timeline

Part A · Lecture

Time	Segment	Min
0:00	Welcome & why ML	10
0:10	Paradigms + Quiz 1	15
0:25	Loss, GD, PyTorch + Quiz 2	20
0:45	Data splits + Quiz 3	10
0:55	Break	5

Part B · Lab 1 and wrap-up

Time	Segment	Min
1:00	Lab setup	5
1:05	Ex 1 · NumPy vectorization	10
1:15	Ex 2 · Pandas cleaning	15
1:30	Ex 3 · Features & heatmap	12
1:42	Lab quiz	8
1:50	Rubric, recap, exit ticket	10

OBJECTIVES

By the end of today you can…

1. Distinguish supervised, unsupervised and reinforcement learning
2. Compute MSE and explain how gradient descent updates parameters
3. Split data into train, validation and test sets
4. Clean and explore a dataset with NumPy, Pandas and Seaborn

WHY ML · 0:00

Rules versus learning

TRADITIONAL PROGRAMMING

You write the rules by hand



MACHINE LEARNING

The algorithm finds the rules from examples



Key takeaway: we show an algorithm examples and let it optimise its internal parameters, so it generalises to unseen examples.

WARM-UP · 2 MIN

Spam detection or sorting a list: which would you rather hand-code, and why?

PARADIGMS · 0:10

The three ML paradigms

Supervised

Labeled data (X, y)

Every input example is paired with a ground-truth label y.

Examples: house prices, spam detection

Unsupervised

Unlabeled data (X)

No targets y. Uncover hidden structure or low-dimensional representations.

Examples: K-Means, PCA, Autoencoders

Reinforcement

Reward-based feedback

An agent acts in an environment and learns a policy from scalar rewards.

Examples: game-playing agents, robot control

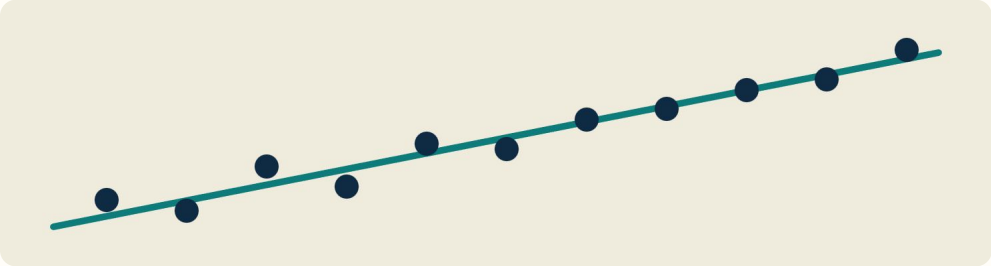
In industry, supervised learning remains the backbone for tabular and visual prediction. Foundation models rely on self-supervised pretraining, a flavour of unsupervised learning.

SUPERVISED LEARNING · 0:14

Regression and classification

Regression

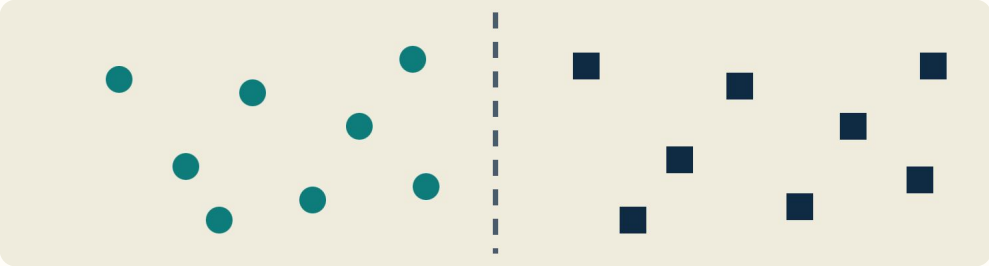
Predict a continuous target.



Example: predicting house prices (USD)

Classification

Predict a discrete class label.



Example: spam vs. non-spam emails

Ask · 1 min: is predicting customer churn regression or classification?

NO LABELS, NO PROBLEM · 0:18

Unsupervised and reinforcement learning

Unsupervised: find structure

The algorithm receives only the inputs X. It groups similar points or compresses the data into fewer dimensions.

K-Means · PCA · Autoencoders

Reinforcement: learn from rewards

An agent takes actions and receives scalar rewards, learning an optimal policy.



Foundation models pretrain with self-supervised learning, a flavour of unsupervised learning.

QUIZ 1 · QUESTIONS 1–3 · 0:20

Check your understanding

QUESTION 1

Which statement best describes machine learning?

A. Hand-writing if/else rules for every case

B. Building systems that learn patterns from data

C. Storing large amounts of data in a database

D. Making code run faster on a GPU

QUESTION 2

A bank has past transactions, each marked “fraud” or “not fraud”. Which setting is this?

A. Unsupervised clustering

B. Reinforcement learning

C. Supervised classification

D. Supervised regression

QUESTION 3

Predicting a house price in USD from its size is a…

A. Classification task

B. Regression task

C. Clustering task

D. Reinforcement task

Think for 30 seconds, vote A to D, then reveal the answers on the next slide.

QUIZ 1 · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: B

Building systems that learn patterns from data

ML learns patterns from examples instead of relying on manually hardcoded rules.

QUESTION 2

Answer: C

Supervised classification

Every example carries a label y, and the label is a discrete class (fraud or not fraud).

QUESTION 3

Answer: B

Regression task

The target, a price in USD, is continuous, so this is regression.

QUIZ 1 · QUESTIONS 4–6 · 0:23

Check your understanding

QUESTION 4 · TRUE OR FALSE

K-Means needs a ground-truth label y for every training example.

A. True

B. False

QUESTION 5

A robot gets a reward each time it moves forward without falling. This is…

A. Supervised learning

B. Unsupervised learning

C. Reinforcement learning

D. Regression

QUESTION 6

Which one is NOT an example of unsupervised learning?

A. K-Means clustering

B. Principal Component Analysis

C. An autoencoder

D. A spam vs. non-spam classifier

Think for 30 seconds, vote, then reveal the answers on the next slide.

QUIZ 1 · ANSWERS 4–6

Answers and explanations

QUESTION 4

Answer: B (False)

K-Means is unsupervised. It receives only the inputs X and looks for structure, with no labels.

QUESTION 5

Answer: C

Reinforcement learning

An agent acts in an environment and learns from scalar rewards.

QUESTION 6

Answer: D

A spam vs. non-spam classifier

Spam detection uses labeled emails, so it is supervised classification.

A dataset is a matrix

Student performance data (illustrative values)

Sample	study_hours	attendance_pct	exam_score (y)
Sample 1	14.2	78.5	71.0
Sample 2	17.9	91.3	84.6
Sample 3	9.6	65.0	58.2
Sample N	…	…	…

```
X.shape → (N, d)
```

N number of samples (rows)

d number of features (columns of X)

y one label per sample, in supervised learning

LOSS FUNCTION · 0:28

Mean Squared Error (MSE)

```
MSE = (1 / N) * sum((y_i - f(x_i))^2)
```

y (true)	f(x) (pred)	error	squared
2	3	+1	1
5	5	0	0
10	8	−2	4
Mean			5 / 3 = 1.67

1. Error: prediction minus true value
2. Square it: no sign cancellation, big misses count more
3. Average over all N samples

OPTIMIZATION · 0:31

Gradient descent in four steps

1
Forward pass
`y_pred = Xw + b`

→

2
Loss
MSE between y_pred and y

→

3
Gradients
dL/dw and dL/db

→

4
Update
```
w -= lr * dw
b -= lr * db
```

Learning rate (lr): the step size, 0.1 in the lecture example.

Epoch: one full cycle of steps 1 to 4. We repeat it 100 times.

The gradient points uphill, so we step the other way.

HANDS-ON CODE · 0:34

Pure NumPy: gradients by hand

```
# y = 3x + 2 + noise; w random, b = 0, lr = 0.1
for epoch in range(100):
    y_pred = np.dot(X, w) + b  # forward pass
    loss = np.mean((y_pred - y) ** 2)  # MSE
    dw = (2 / len(X)) * np.dot(X.T, (y_pred - y))
    db = (2 / len(X)) * np.sum(y_pred - y)
    w -= lr * dw  # update
    b -= lr * db
print(f"Learned Weight: {w[0,0]:.4f}, Learned Bias: {b[0,0]:.4f}")
```

Analytical gradients of the MSE: you derive dL/dw and dL/db yourself.

OUTPUT (SEED 42)

```
Learned Weight: 2.5870
Learned Bias: 2.2072
```

True values: w = 3, b = 2

DISCUSS · 2 MIN

Why is the result not at 3 and 2 yet?

HANDS-ON CODE · 0:37

PyTorch: autograd does the calculus

```
model = torch.nn.Linear(in_features=1, out_features=1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
criterion = torch.nn.MSELoss()

for epoch in range(100):
    optimizer.zero_grad()  # clear old gradients
    predictions = model(X_tensor)  # forward pass
    loss = criterion(predictions, y_tensor)  # compute MSE
    loss.backward()  # autograd computes gradients
    optimizer.step()  # update parameters
```

COMMON PITFALL

Call optimizer.zero_grad() before loss.backward(). Gradients accumulate by default, so uncleared buffers make training diverge.

WHY IT HELPS

Autograd builds a dynamic computational graph, so you never derive multivariable gradients by hand.

QUIZ 2 · QUESTIONS 1–3 · 0:40

Check your understanding

QUESTION 1

A dataset has N samples and d features per sample. What is the shape of the feature matrix X?

A. d × N

B. N × d

C. N × N

D. d × d

QUESTION 2

Predictions are [2, 4] and true labels are [1, 6]. What is the MSE?

A. 1.5

B. 5.0

C. 2.5

D. 0.5

QUESTION 3

Why does MSE square the errors?

A. So errors do not cancel and large errors weigh more

B. So positive and negative errors can cancel out

C. So the loss is always exactly zero

D. So that no model is needed

Work Question 2 on paper: write down each error, square it, then average.

QUIZ 2 · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: B

N × d

One row per sample and one column per feature. In the lecture code, X has shape (100, 1).

QUESTION 2

Answer: C

MSE = 2.5

Errors are +1 and −2. Squares are 1 and 4. The mean is (1 + 4) / 2 = 2.5.

QUESTION 3

Answer: A

Errors do not cancel, large errors weigh more

Squaring removes the sign, so +1 and −1 no longer sum to zero, and big misses are penalised more.

QUIZ 2 · QUESTIONS 4–6 · 0:43

Check your understanding

QUESTION 4

In gradient descent with learning rate lr, which update is correct?

A. w = w + lr × gradient

B. w = lr ÷ gradient

C. w = w − lr × gradient

D. w = a random number

QUESTION 5

What does optimizer.zero_grad() do in a PyTorch training loop?

A. Sets all model weights to zero

B. Stops the training loop

C. Sets the loss to zero

D. Clears gradients left from the previous step

QUESTION 6

Which is the correct order for one PyTorch training step?

A. zero_grad, forward, loss, backward, step

B. forward, loss, step, backward

C. backward, zero_grad, forward, step

D. step, forward, loss, zero_grad

Hint for Question 6: look back at the training loop on the PyTorch slide.

QUIZ 2 · ANSWERS 4–6

Answers and explanations

QUESTION 4

Answer: C

w = w − lr × gradient

Step against the gradient to reduce the loss. In code: w -= lr * dw.

QUESTION 5

Answer: D

Clears gradients from the previous step

Gradients accumulate by default, so uncleared buffers can make training diverge.

QUESTION 6

Answer: A

zero_grad, forward, loss, backward, step

Clear, predict, measure the loss, backpropagate, then update the parameters.

GENERALIZATION · 0:45

Train, validation and test sets



70–80%

Training set

Used to update model parameters via gradient descent.

10–15%

Validation set

Used to tune hyperparameters (learning rate, regularization strength, depth) and to stop early.

10–15%

Test set

Kept isolated until the final evaluation. Gives an unbiased estimate of real-world performance.

HANDS-ON CODE · 0:48

Splitting with scikit-learn

```
# 1) hold out 15% for the test set
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42)

# 2) 0.1765 of the remaining 85% is about 15% of all data
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.1765, random_state=42)
```

WHY 0.1765?

0.15 ÷ 0.85 ≈ 0.1765, so the validation set is about 15% of the full data.

OUTPUT FOR N = 100

```
Train size: 69
Val size: 16
Test size: 15
```

Rounding gives 69 / 16 / 15, close to 70 / 15 / 15.

QUIZ 3 · QUESTIONS 1–3 · 0:50

Check your understanding

QUESTION 1

Which subset is used to tune hyperparameters and decide when to stop early?

A. Test set

B. Training set

C. Validation set

D. All of them equally

QUESTION 2 · TRUE OR FALSE

You may check the test set repeatedly while tuning, as long as you never train on it.

A. True

B. False

QUESTION 3

You split 1,000 samples 70 / 15 / 15. How many are train / val / test?

A. 800 / 100 / 100

B. 600 / 200 / 200

C. 700 / 200 / 100

D. 700 / 150 / 150

Last checkpoint of the lecture: vote, then reveal.

QUIZ 3 · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: C

Validation set

It tunes the learning rate, regularization strength and depth, and supports early stopping.

QUESTION 2

Answer: B (False)

The test set stays isolated until the final evaluation. Repeated peeking makes the estimate biased.

QUESTION 3

Answer: D

700 / 150 / 150

70% of 1,000 is 700. The other 300 split evenly into 150 validation and 150 test.



BREAK · 0:55

Five-minute break

Back at 1:00 for Lab 1. Open Python 3.10 or higher and a new Jupyter notebook, then install the packages:

```
pip install numpy pandas matplotlib seaborn scikit-learn
```

LAB 1 · 1:00

Python data science foundations and EDA

Objective: build an exploratory data analysis pipeline on a synthetic student performance dataset, compute key statistical summaries and generate plots.

EXERCISE 1 · 10 MIN

NumPy vectorization

Compare a Python loop with array operations on 1,000,000 samples.

25% of the grade

EXERCISE 2 · 15 MIN

Pandas cleaning

Find missing values in 200 students and impute them with medians.

35% of the grade

EXERCISE 3 · 12 MIN

Features and heatmap

Create high_performer, compute correlations and plot a Seaborn heatmap.

30% of the grade

The remaining 10% rewards code organization and clear Markdown.

LAB SETUP · 1:00 · 5 MIN

Prerequisites and environment

1. Use Python 3.10 or higher
2. Install the packages with pip (command on the break slide)
3. Print the version numbers to verify

CHECKPOINT

Raise a hand when every version prints without errors.

```
import sys, numpy, pandas, matplotlib, seaborn, sklearn

print(sys.version)
print(numpy.__version__, pandas.__version__)
print(matplotlib.__version__, seaborn.__version__)
print(sklearn.__version__)
```

PyTorch is not needed for Lab 1.

EXERCISE 1 · 1:05 · 10 MIN

NumPy vectorization vs. loops

```
N = 1_000_000
y_true = np.random.rand(N)
y_pred = np.random.rand(N)

# 1) pure Python loop (time it with time.time())
python_mse = sum((t - p) ** 2 for t, p in zip(y_true, y_pred)) / N
# 2) vectorized NumPy (time it the same way)
numpy_mse = np.mean((y_true - y_pred) ** 2)

speedup = python_duration / numpy_duration
```

PREDICT FIRST · 1 MIN

How many times faster will NumPy be: 2×, 10× or 100×?

RECORD YOUR RESULT

The lab page expects 50–100×, but the gap depends on your machine (a quick test run gave about 4×). Both MSE values should match, near 1/6 ≈ 0.167.

EXERCISE 2A · 1:15 · 7 MIN

Build and inspect the dataset

```
np.random.seed(42)
n_students = 200
data = {
  "student_id": [f"STU_{i:03d}" for i in range(1, n_students + 1)],
  "study_hours": np.random.normal(15, 4, n_students).round(1),
  "attendance_pct": np.random.uniform(60, 100, n_students).round(1),
  "exam_score": np.random.normal(75, 12, n_students).round(1),
}
df = pd.DataFrame(data)

mask_study = np.random.rand(n_students) < 0.08
mask_exam = np.random.rand(n_students) < 0.05
df.loc[mask_study, "study_hours"] = np.nan
df.loc[mask_exam, "exam_score"] = np.nan
print(df.isnull().sum())
```

MISSING VALUES (SEED 42)

Column	NaN
student_id	0
study_hours	14
attendance_pct	0
exam_score	11

NaNs are injected at random with probabilities 8% and 5%.

EXERCISE 2B · 1:22 · 8 MIN

Impute missing values with the median

```
# Impute missing study_hours with the median
study_median = df["study_hours"].median()
df["study_hours"] = df["study_hours"].fillna(study_median)

# Impute missing exam_score with the median
exam_median = df["exam_score"].median()
df["exam_score"] = df["exam_score"].fillna(exam_median)

print(df.isnull().sum())
```

PREDICT FIRST · 1 MIN

What will df.isnull().sum() print after imputation?

BEST PRACTICE

For skewed features prefer the median. The values 10, 11, 12, 13, 100 have mean 29.2 but median 12.

OUR RUN

Median study_hours is 14.8 and median exam_score is 73.9. Afterwards every missing count is 0.

EXERCISE 3 · 1:30 · 12 MIN

Feature engineering and heatmap

```
df["high_performer"] = (df["exam_score"] >= 80).astype(int)

corr_matrix = df[["study_hours", "attendance_pct",
                  "exam_score", "high_performer"]].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="Blues",
            fmt=".2f", linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```

DISCUSS · 3 MIN

The lab page expects strong positive correlations for study_hours and attendance_pct. These columns were generated independently. What do you see?

CORRELATIONS IN OUR RUN (SEED 42)

	SH	AT	ES	HP
SH	1.00	−0.05	−0.03	−0.02
AT	−0.05	1.00	0.01	0.00
ES	−0.03	0.01	1.00	0.77
HP	−0.02	0.00	0.77	1.00

SH = study_hours, AT = attendance_pct, ES = exam_score, HP = high_performer.

The only strong cell exists because HP is derived from ES.

LAB QUIZ · QUESTIONS 1–3 · 1:42

Lab check: NumPy and Pandas

QUESTION 1

Why is np.mean((y_true - y_pred) ** 2) faster than the Python loop?

A. It uses only a small sample of the data

B. It works on whole C-backed arrays at once

C. Python lists cannot store floats

D. It skips the squaring step

QUESTION 2

Why prefer the median over the mean when imputing a skewed feature?

A. The median is always larger than the mean

B. The mean cannot be computed with NaN present

C. The median is robust to outliers

D. The median creates new missing values

QUESTION 3

With seed 42, study_hours has 14 NaN and exam_score has 11. After median imputation, df.isnull().sum() shows…

A. 14 and 11, unchanged

B. 0 only for study_hours

C. 25 in total

D. 0 for every column

Answer from what you just ran in the notebook, then reveal.

LAB QUIZ · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: B

It works on whole C-backed arrays at once

Vectorized operations avoid per-element Python overhead.

QUESTION 2

Answer: C

The median is robust to outliers

Outliers pull the mean, not the median. Pandas skips NaN when computing either one.

QUESTION 3

Answer: D

0 for every column

fillna replaced all 14 and 11 missing values, so no NaN remains.

LAB QUIZ · QUESTIONS 4–6 · 1:46

Lab check: features and correlation

QUESTION 4

What does (df["exam_score"] >= 80).astype(int) create?

A. A text label for each student

B. 1 if the score is at least 80, otherwise 0

C. A copy of exam_score

D. A continuous feature from 0 to 1

QUESTION 5

study_hours and exam_score were generated independently. Which correlation do you expect?

A. Close to +1

B. Close to −1

C. Close to zero

D. Exactly +1

QUESTION 6 · TRUE OR FALSE

high_performer correlates strongly with exam_score because it is derived from exam_score.

A. True

B. False

Check your own heatmap values before voting.

LAB QUIZ · ANSWERS 4–6

Answers and explanations

QUESTION 4

Answer: B

1 if the score is at least 80, otherwise 0

The comparison gives True or False, and astype(int) turns that into 1 or 0.

QUESTION 5

Answer: C

Close to zero

Independent random columns carry no linear relationship. Our run gave −0.03.

QUESTION 6

Answer: A (True)

It is computed from exam_score by a threshold, so the correlation is high by construction (0.77 in our run).

WRAP-UP · 1:50

Deliverables and grading

Submit one Jupyter Notebook (.ipynb)

- ✓ Full executable Python cells with code and output
- ✓ Written responses interpreting the feature correlation matrix
- ✓ Clean code following PEP 8 guidelines

Grading criterion	Weight
NumPy vectorization execution	25%
Pandas data cleaning and imputation	35%
Feature engineering and Seaborn visualization	30%
Code organization and Markdown clarity	10%

RECAP · 1:55

What we covered today

1. Supervised, unsupervised and reinforcement learning differ in their data and feedback.
2. Loss functions such as MSE steer parameter updates through gradients.
3. PyTorch autograd replaces hand-derived gradients. Remember zero_grad().
4. Keep the test set untouched. Clean, vectorize and explore data before modelling.

EXIT TICKET · 2 MIN

In one sentence: why must the test set stay untouched until the final evaluation?

NEXT

Lecture 2: Linear Models and Regularization (L1 Lasso and L2 Ridge). Submit your Lab 1 notebook before then.